

What is programming?

⇒ programming is the process of writing instructions that tell a computer what to do!

- eg:
1. Boil water
 2. Add tea leaves
 3. Add sugar
 4. Add milk
 5. Serve

These instructions are called a program.

A programmer who writes these instructions is called a Programmer.

How Computer understands Instructions?

⇒ Computer understands only binary language (0 & 1)

Since humans cannot write huge binary no.s, programming languages were invented!

What is programming language?

⇒ A programming language is a language used by humans to communicate with computers.

eg: instead of writing : 101011010010
we write : `System.out.println("Hello");`

Evolution of Programming languages

⇒ 1. Machine Language : Lowest level language.
Used only 0 & 1.

Adv: Fastest
No translator needed.

Disadv: Extremely difficult
error-prone

eg: 1011010100111
Impossible to remember.

Machine dependent

2. Assembly language : Instead of binary, use mnemonics
eg: ADD MOV DIV
 SUB MUL

Instead of : 1011 0011

we write: ADD A, B

Adv: Much easier

Disadv: Needs an Assembler

Adv: Easier than ML
Faster execution

Disadv: Still difficult
Machine dependent.

3. High level language : Closer to human language.
eg: C, C++, Java, Python, Kotlin

eg: int a = 10;
int b = 20;

System.out.println(a+b);

Adv: Easy
Readable
portable
easy debugging

Disadv: Slightly slower
because translation
is required.

Translator : Compiler / Interpreter

High level language

• Closer to humans
eg: Java, Python, C++

Adv: Easy, readable,
portable

Disadv: Need translation

Low level language

• closer to hardware
eg: ML, Assembly language

Adv: Better hardware control,
faster

disadv: Difficult

Types of programming languages (Programming lang)

1. Procedural programming

→ series of well structured steps & procedures to
compose a program.

→ contains a systematic order of statements &
& commands to complete a task

Adv: Easy & structured

Disadv: Difficult for low

2. Assembly language : Instead of binary, use mnemonics
eg: ADD MOV DIV
SUB MUL

Instead of : 101100111

we write: ADD A, B

• Much easier

• Needs an Assembler

Adv: Easier than ML
Faster execution

Disadv: Still difficult
Machine dependent.

3. High level language : Closer to human language.

eg: C, C++, Java, Python, Kotlin

eg: int a = 10;
int b = 20;

System.out.println(a+b);

Adv: Easy

Readable

portable

easy debugging

Disadv: Slightly slower
because translation
is required.

Translator: Compiler / Interpreter

High level language

• Closer to humans

eg: Java, Python, C++

Adv: Easy, readable,
portable

Disadv: Need translation.

Low level language

• closer to hardware

eg: ML, Assembly language

Adv: Better hardware control,
faster

disadv: Difficult

Types of programming languages (Programming paradigms)

1. Procedural Programming

- series of well structured steps & procedures to compose a program.
- contains a systematic order of statements, funcⁿ & commands to complete a task

Adv: Easy, structured

Disadv: Difficult for large software

2. Functional Programming

- Writing a program only in pure functions i.e., never modify variables but only create new ones as an output.
- Used in a situation where we have to perform lots of different operations on the same set of data like ML.

eg:

$Salary = salary + 1000 \times$

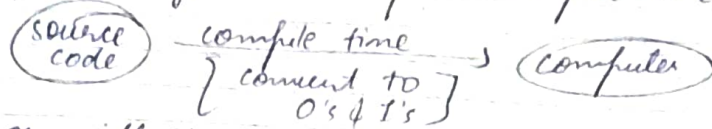
$newSalary = salary + addBonus(salary)$

3. Object-Oriented Programming

- revolves around objects
- code + data = objects
- developed to make it easier to develop, debug, reuse & maintain.

Java $\implies$ object oriented but also supports procedural programming.

→ perform type checking at compile time



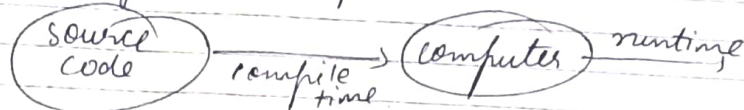
- error will show at compile time
- declare datatype before use

int a = 10

→ More control over the program.

Dynamic languages:

→ perform type checking at runtime



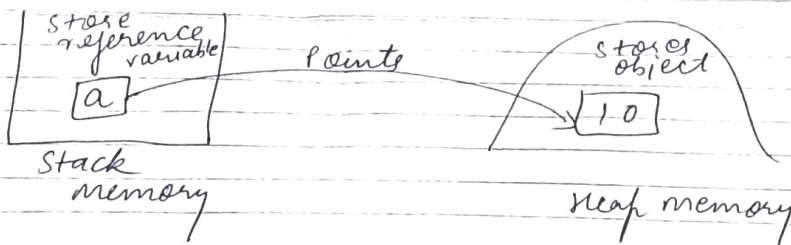
- error might not show till programs run
- no need to declare datatype of variables

a = 10 { language by itself figures out datatype }

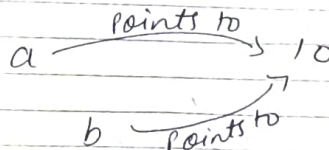
→ saves time in writing code but might give error at runtime.

Memory Management

a = 10 ← object
reference variable



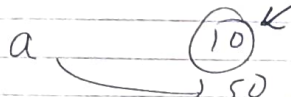
Now suppose,
a = 10
b = a



- More than one reference variable can point towards one object.
- If any of the reference variable changes, the object then it is changed for all reference variable that points towards same object.

Now initially,

Then, a = 10
a = 50



object with no reference variable

removed by garbage collector

compiler

→ A compiler converts the entire program into machine code before execution

eg: source code → compiler → Machine code → Run

eg: C, C++

Adv: fast execution, error shown together.
Disadv: Need recompilation after changes.

Interpreter

→ Translate one line at a time
eg: line 1 → execute → line 2 → execute
eg: Python, JavaScript.

Adv: easy debugging

Disadv: slower

Java uses both

Java code → compiler (javac) → Byte code (.class),

↓
JVM

Machine
code

Interpreters
+
JIT compiler

Algorithm

→ An algorithm is a set of instruction for solving a particular problem in finite amount of time with finite amount of data.

→ step by step procedure to solve a problem.

Characteristics

1. Finite: algo. must terminate in finite times
2. Unambiguous (Definiteness): every single instruction must be perfectly clear & precise
3. Correct: produce accurate & intended results
4. Efficient (Effectiveness): less time & space to execute
5. Inputs: can take 0 or more inputs.
6. Outputs: must produce atleast 1 output.

Flowchart

→ Graphical representation of an algorithm
Instead of writing steps we draw them.

Symbols.



Oval

⇒ Start / End



Rectangle

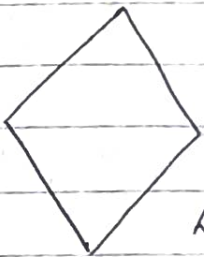
⇒ Process eg: $sum = a + b$



Input/Output

⇒ Input / Output

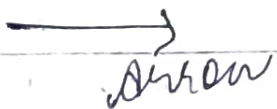
eg: Input Name
Print Result



Diamond

⇒ Decision eg:

Marks > 40 ?
Yes / No



Arrow

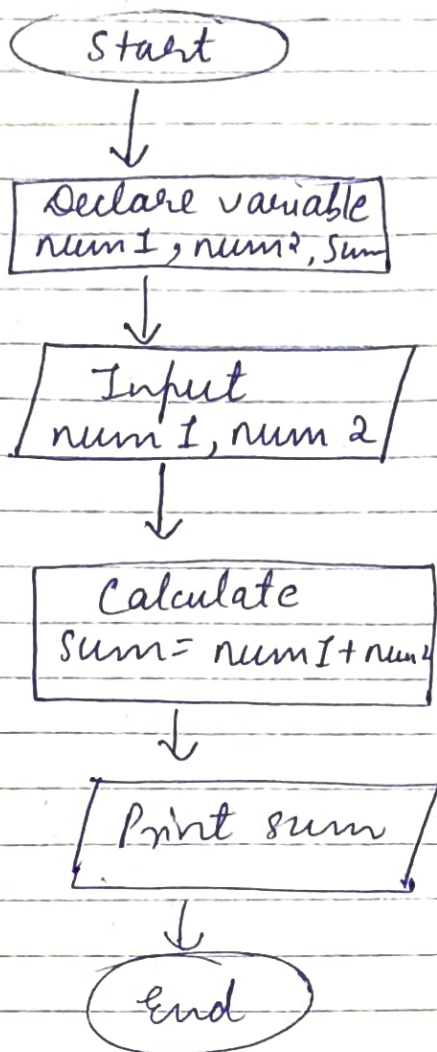
⇒ ~~Arrow~~ Flow direction
(show execution sequence)

Pseudocode

A rough program written in simple English
No syntax rules

Questions

- 1) Take two no.s and print the sum of both
→ Flowchart



Algo.

1. Start the process
2. Declare three variables num1, num2, sum
3. Read (Input) the value of num 1
4. Read the value of num 2
5. Calculate the addition (sum) by adding num1 & num2
6. Store the result in sum variable
7. Display (Print) the value of sum
8. End the process.

Pseudo.

START

DECLARE num1, num2, SUM as
Numbers

DISPLAY "Enter the first no.:"

INPUT num1

DISPLAY "Enter the second no.:"

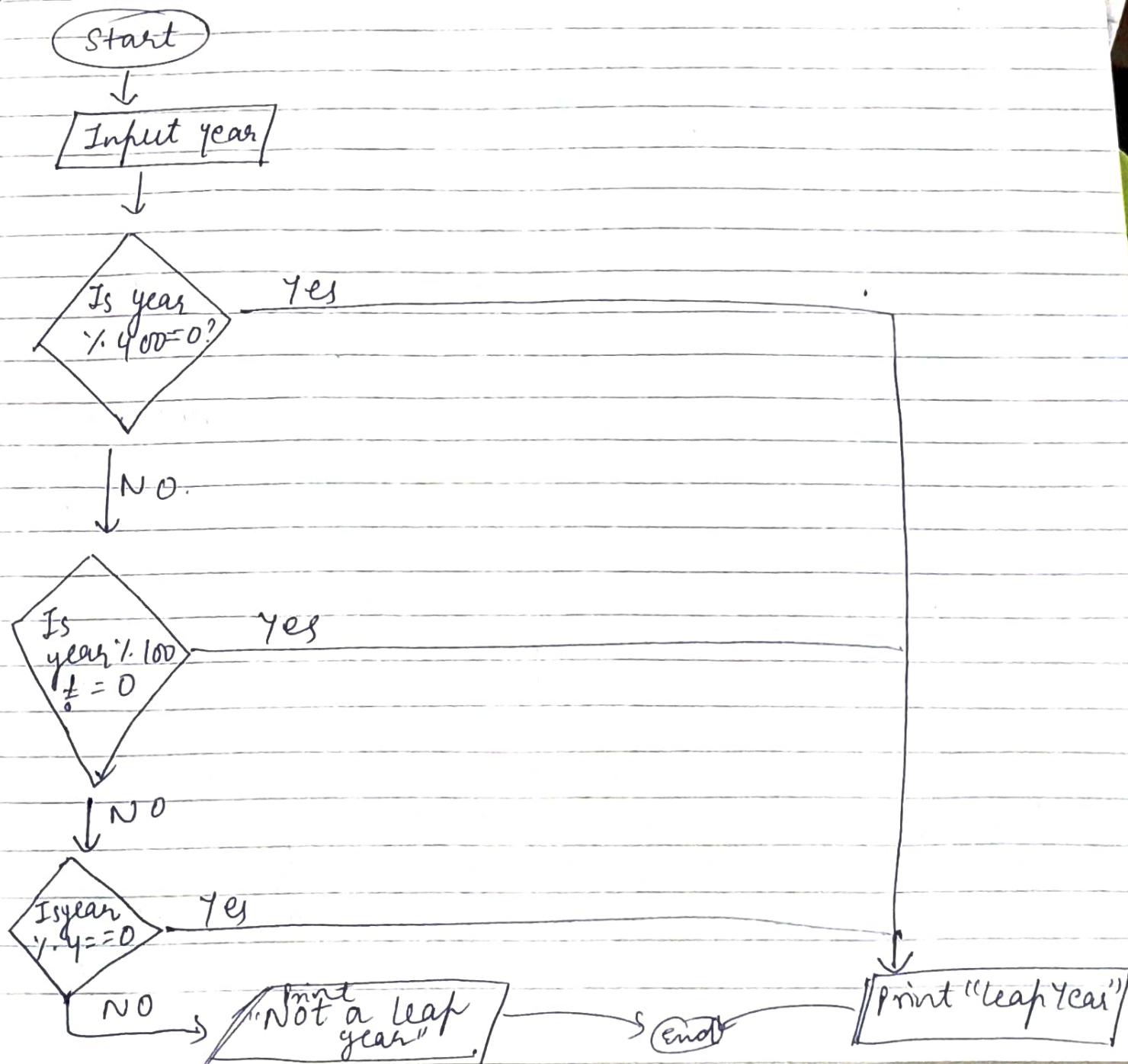
INPUT num2

SET sum = num1 + num2

DISPLAT "The sum is:", sum

END

2) Input a year and find whether it is a leap year or not
⇒ Flowchart



Algo.

1. Start
2. Input the year
3. Check if year divisible by 400. If yes go to step 6
4. Check if year divisible by 100. If yes go to step ~~7~~
5. Check if year divisible by 4. If yes go to step 6 else
Step 4.
6. Print "Leap year" & go to step 8
7. Print "Not a leap year" & go to step 8
8. End.

Pesudo:

1. START

DECLARE year as INTEGER

DISPLAY "Enter a year: "

INPUT year

IF (year % 400 == 0) THEN

DISPLAY year, "is a leap year"

ELSE IF (year % 100 == 0) THEN

DISPLAY year, "not a leap year"

ELSE IF (year % 4 == 0) THEN

DISPLAY year, "is a leap year"

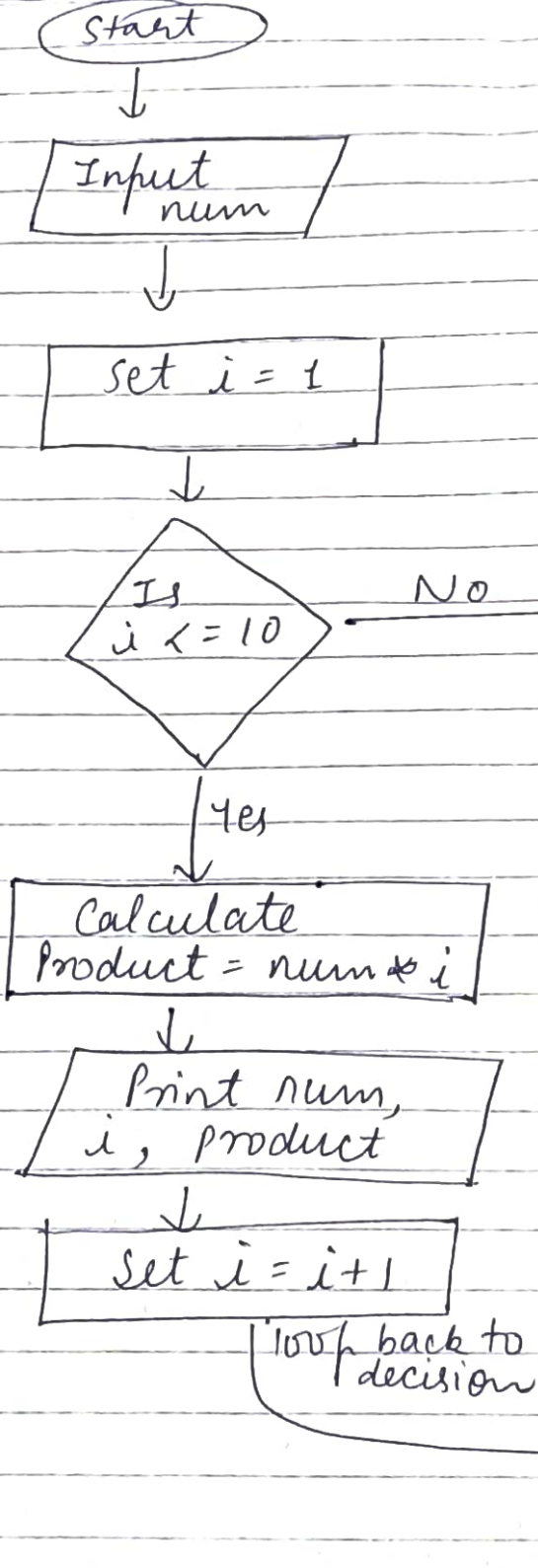
ELSE

DISPLAY year, "is not a leap year"

END

3) Take a no. as input & print the multiplication table for it.

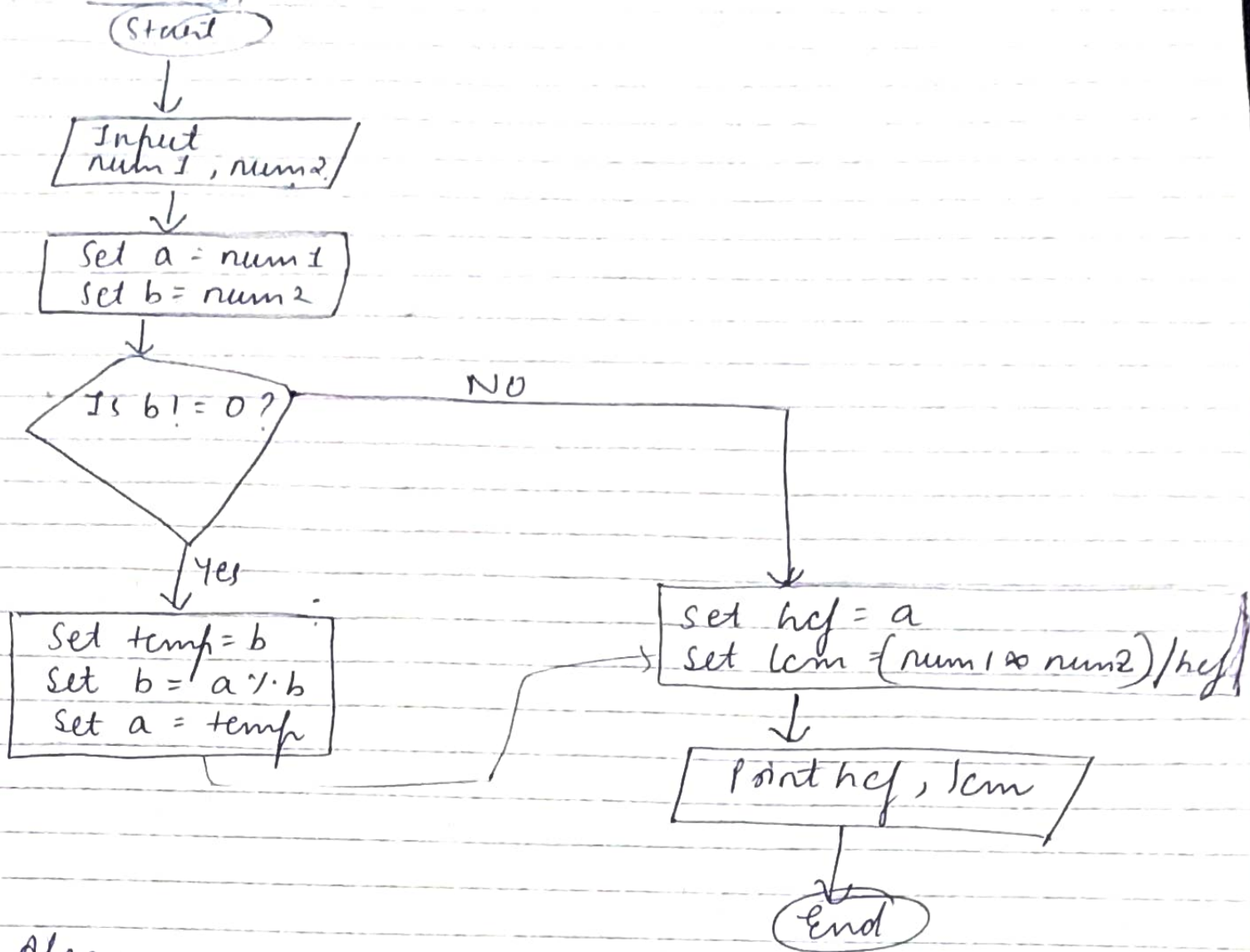
⇒ Flowchart



Algo.

1. Start
2. Declare variable num & i
3. Input value of num
4. Set initial value of i to 1
5. Check if i is less than or equal to 10. If false go to step 9
6. Calculate product = num * i
7. Print the result in format: num * i = product
8. Increment i by 1 ($i = i + 1$) & go back to step 5
9. Stop.

4) Take 2 no.s as input & find their HCF & LCM
 → Flowchart.

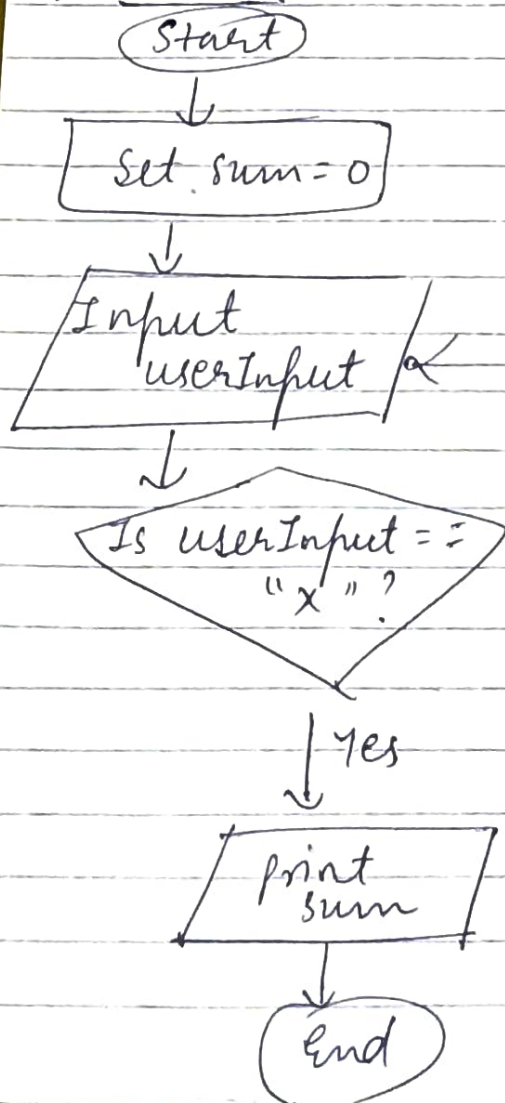


Algo.

1. Start
2. Declare num1, num2, a, b, temp, hcf, lcm.
3. Input the values of num1 & num2
4. Copy the original values : Set a = num1 & b = num2
5. Repeat while b is not equal to 0 :
 1. Set temp = b
 2. Set b = a % b
 3. Set a = temp
6. Set hcf = a
7. Calculate LCM : $lcm = (num1 \times num2) / hcf$
8. Print value of hcf & lcm
9. Stop.

5) Keep taking no.s as inputs till the user enters 'x', after that print sum of all.

=> Flowchart.



Algo.

1. Start
2. Declare variable : sum = 0 & info userInput (To store text input)
3. Repeat
 1. Input a value & store it in userInput
 2. Check userInput is equal to "x". If yes exist
 3. convert userInput from text to a numeric value
 4. Add the numeric value to sum (sum = sum + value)
4. Print sum
5. Stop.